📦 AWS Storage — Category 4: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** S3, EBS, EFS, FSx, Storage Gateway, Lifecycle Policies, Replication, Presigned URLs, Object Lock, Multipart Upload, S3 Select, RAID, io2 Block Express

---

📋 Table of Contents

1. 4.1 S3 — Buckets, Objects, Storage Classes, Versioning
2. 4.2 EBS — Volume Types, Snapshots, Encryption
3. 4.3 S3 — Lifecycle Policies, Replication, Presigned URLs
4. 4.4 EFS — Use Cases, Performance Modes, Mount Targets
5. 4.5 S3 Security — Bucket Policies, ACLs, Block Public Access, S3 Object Lock
6. 4.6 S3 — Event Notifications, S3 Select, Multipart Upload
7. 4.7 EBS — RAID Configurations, io2 Block Express
8. 4.8 FSx — Windows, Lustre, NetApp ONTAP
9. 4.9 Storage Gateway — Types and Use Cases

4.1 S3 — Buckets, Objects, Storage Classes, Versioning

🟢 What It Is in Simple Terms

S3 (Simple Storage Service) is AWS's object storage service. Think of it as an infinitely scalable hard drive in the cloud where you store files (called objects) in containers (called buckets). Unlike a file system, there's no folder hierarchy — just a flat namespace with key-value pairs. It's the backbone of almost every AWS architecture.

🔍 Why It Exists / What Problem It Solves

Traditional file servers had capacity limits, single points of failure, and geographic constraints. S3 gives you unlimited storage, 11 nines of durability (99.999999999%), automatic replication across 3+ AZs, and global accessibility — without managing a single server.

⚙️ How It Works Internally

```
S3 Internal Architecture:

S3 is NOT a file system. It's a key-value store.

Key   = "images/profile/user123.jpg"  (looks like a path, is NOT)
Value = the actual bytes of the object
Metadata = content-type, size, custom headers, tags

Bucket namespace:
├── Bucket names are GLOBALLY unique across ALL AWS accounts
├── Bucket lives in ONE region (you choose)
├── But bucket name must be unique across ALL regions, ALL accounts
└── DNS-based: my-bucket.s3.amazonaws.com

S3 Durability vs Availability:
├── Durability: 99.999999999% (11 nines)
│   = If you store 10M objects, expect to lose 1 every 10,000 years
│   = Achieved by storing data redundantly across 3+ AZs
│
└── Availability: 99.99% (Standard) = ~52 min downtime/year
    (Different from durability — this is about access, not data loss)

Object size limits:
├── Single PUT: up to 5 GB
├── Max object size: 5 TB
└── Multipart upload: required for > 5GB, recommended for > 100MB
```

🧩 Buckets

```
Bucket properties:
├── Name: globally unique, 3-63 chars, lowercase, no underscores
├── Region: fixed at creation
├── Versioning: enabled/disabled/suspended
├── Encryption: SSE-S3, SSE-KMS, SSE-C, DSSE-KMS
├── Access: bucket policy, ACL, Block Public Access
├── Replication: cross-region or same-region
├── Logging: access logs to another S3 bucket
├── Events: trigger Lambda/SQS/SNS on object operations
└── Tags: for cost allocation

⚠ Bucket names must be valid DNS labels:
   my_bucket → INVALID (underscore not allowed)
   My-Bucket → INVALID (uppercase not allowed)
   my-bucket → VALID

Object key naming:
S3 used to have performance issues with sequential prefixes
(e.g., 2024-01-01/file1.jpg, 2024-01-02/file1.jpg)
because same prefix = same partition = hotspot.
```

```
Modern S3 auto-scales to 3,500 PUT/s and 5,500 GET/s
PER PREFIX — no longer need to randomize prefixes.
But still good practice to understand prefix-based partitioning.
```

🧩 Storage Classes

```
S3 Storage Classes — cost vs retrieval speed tradeoff:

┌──────────────────────┬─────────┬──────────────┬──────────────────┐
│ Storage Class        │$/GB/mo  │ Retrieval    │ Use Case         │
├──────────────────────┼─────────┼──────────────┼──────────────────┤
│ S3 Standard          │$0.023   │ ms           │ Frequently accessed │
│ S3 Intelligent-Tiering │$0.023   │ ms-mins      │ Unknown access patt │
│ S3 Standard-IA       │$0.0125  │ ms           │ Infrequent, rapid   │
│ S3 One Zone-IA       │$0.01    │ ms           │ Infrequent, 1 AZ ok │
│ S3 Glacier Instant   │$0.004   │ ms           │ Archive, quick fetch │
│ S3 Glacier Flexible  │$0.0036  │ mins-hours   │ Archive, no rush    │
│ S3 Glacier Deep Archive │$0.00099│ hours       │ Long-term archive   │
└──────────────────────┴─────────┴──────────────┴──────────────────┘

⚠ Key costs beyond storage:
Standard-IA, One Zone-IA, Glacier Instant:
- Minimum storage duration: 30 days (charged even if deleted earlier)
- Retrieval fee: per GB retrieved
- Min object size: 128 KB (smaller objects charged as 128 KB)

Glacier Flexible:
- Minimum storage duration: 90 days
- Retrieval tiers:
  Expedited: 1-5 min,  ~$0.03/GB
  Standard:  3-5 hrs,  ~$0.01/GB
  Bulk:      5-12 hrs, ~$0.0025/GB

Glacier Deep Archive:
- Minimum storage duration: 180 days
- Standard retrieval: 12 hours
- Bulk retrieval: 48 hours

S3 Intelligent-Tiering:
├── Automatically moves objects between tiers based on access
├── No retrieval fees (monitoring fee instead: $0.0025/1000 objects)
├── Tiers: Frequent, Infrequent (30 days), Archive Instant (90 days),
│         Archive Access (90-270 days), Deep Archive (180+ days)
└── Best for unpredictable access patterns
```

🧩 Versioning

```
What versioning does:
Every PUT/DELETE creates a new version.
Old versions are preserved.

Without versioning:
PUT image.jpg → stores image.jpg
PUT image.jpg → OVERWRITES image.jpg (old version gone)
DELETE image.jpg → GONE FOREVER

With versioning:
PUT image.jpg    → version ID: abc123
PUT image.jpg    → version ID: def456 (abc123 still exists)
DELETE image.jpg → adds DELETE MARKER (both versions still exist!)
                   (object appears deleted but isn't)

To truly delete with versioning:
DELETE image.jpg?versionId=abc123 → permanently removes that version

Version IDs:
├── null → object uploaded before versioning was enabled
└── UUID → e.g., "3sL4kqtJlcpXrQ..."

MFA Delete:
├── Requires MFA to permanently delete versions
└── Extra protection against accidental/malicious deletions

⚠ Versioning triples/quadruples your storage costs if objects
   change frequently. Use lifecycle rules to expire old versions.

Versioning states:
Unversioned → Enabled → Suspended
(Cannot go back to Unversioned after enabling)
(Suspended: new objects get null version, old versions preserved)
```

```
# Enable versioning
aws s3api put-bucket-versioning \
  --bucket my-bucket \
  --versioning-configuration Status=Enabled

# List versions of an object
aws s3api list-object-versions \
  --bucket my-bucket \
  --prefix image.jpg

# Delete specific version
aws s3api delete-object \
  --bucket my-bucket \
  --key image.jpg \
  --version-id abc123

# Restore previous version (copy old version over current)
```

```
aws s3api copy-object \
  --bucket my-bucket \
  --copy-source "my-bucket/image.jpg?versionId=abc123" \
  --key image.jpg
```

💬 Short Crisp Interview Answer

"S3 is AWS's object storage — infinitely scalable, 11 nines of durability, regional but globally accessible. Objects are stored in buckets with a flat key-value structure. Storage classes let you trade cost for retrieval speed — Standard for frequent access, Intelligent-Tiering for unpredictable patterns, Standard-IA for infrequent but rapid access, and Glacier tiers for archive. Versioning preserves every version of an object and turns deletes into delete markers rather than actual deletions — it's your first line of defense against accidental overwrites or deletions. The gotchas: bucket names are globally unique, IA classes have minimum storage duration and retrieval fees, and versioning can silently multiply your storage costs."

🏭 Real World Production Example

```
Media platform storing user uploads:

Bucket structure:
s3://media-platform-prod/
├── uploads/raw/       → Standard (just uploaded, being processed)
├── uploads/processed/ → Standard-IA (processed, rarely re-accessed)
├── thumbnails/        → Standard (frequently served via CloudFront)
└── archive/           → Glacier Deep Archive (legal hold, 7yr retention)

Lifecycle policy:
uploads/raw/*:
  → After 1 day:    transition to Standard-IA
  → After 90 days:  transition to Glacier Flexible
  → After 7 years:  expire (delete)

thumbnails/*:
  → After 30 days without access: Intelligent-Tiering
  (handles viral content — old thumbnails rarely accessed)

Versioning: ENABLED on all prefixes
MFA Delete: ENABLED (only ops team lead has MFA device)

Cost result:
- 1PB total storage
- 80% in archive classes
- Monthly bill: $500 vs $23,000 if all Standard
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Bucket name global uniqueness	Unique across ALL accounts worldwide
Durability ≠ Availability	11 nines durability (data loss) ≠ 99.99% availability (access)
Versioning costs	Every version stored separately. Old versions cost money
Delete marker ≠ deletion	With versioning, DELETE adds a marker. Data still exists
IA minimum duration	30 days minimum. Delete at day 15 = charged for 30
IA retrieval fees	Per-GB retrieval cost on top of storage. Not free to GET
One Zone-IA risk	Data in single AZ. AZ failure = data loss
Intelligent-Tiering small objects	Objects < 128KB not moved to cheaper tiers, still charged monitoring fee

🔗 Connections to Other AWS Concepts

```
S3 connects to:
├── CloudFront    → CDN origin (OAC for private content)
├── Lambda        → Event notifications, triggered processing
├── SNS/SQS       → Event notification destinations
├── EventBridge   → Richer event routing from S3 events
├── Athena        → Query S3 data with SQL
├── Glue          → ETL from S3 data
├── KMS           → Server-side encryption keys
├── IAM           → Identity-based + resource-based policies
├── VPC Endpoints → Private S3 access (Gateway endpoint)
├── Replication   → CRR to other region for DR
├── Object Lock   → WORM compliance
└── Storage GW    → On-prem to S3 bridge
```

4.2 EBS — Volume Types, Snapshots, Encryption

🟢 What It Is in Simple Terms

EBS (Elastic Block Store) is a network-attached block storage drive for EC2 instances — like a hard drive that lives in AWS's network instead of inside the physical server. You can attach it to an EC2 instance, format it with a filesystem, and use it like any disk.

🔍 Why It Exists / What Problem It Solves

EC2 instances need persistent storage that survives reboots and can be detached/reattached. Instance store (local NVMe) is fast but ephemeral — data is lost when instance stops. EBS gives you persistent block storage that persists independently of the instance lifecycle.

⚙️ How It Works Internally

```
EBS Architecture:

EC2 Instance                    AWS Network
┌──────────────┐                ┌──────────────────────┐
│   Your App   │                │   EBS Volume         │
│      ↓       │  Network I/O   │   (replicated within │
│  /dev/xvda   │◄──────────────►│    AZ automatically) │
│  (block dev) │                └──────────────────────┘
└──────────────┘

Key properties:
├── Network-attached (not inside the physical server)
├── AZ-specific: volume lives in one AZ, EC2 must be in same AZ
├── Can ONLY be attached to one EC2 at a time
│   (except io1/io2 Multi-Attach)
├── Persists independently of EC2 lifecycle
├── Automatically replicated within its AZ
└── Can be up to 64 TB (io2 Block Express)

⚠ Because EBS is network-attached, I/O goes over the network.
   EBS performance is bounded by:
   1. Volume type's IOPS/throughput limits
   2. Instance's EBS bandwidth limit (varies by instance type)
   Nitro instances dedicate separate hardware to EBS I/O.
```

🧩 Volume Types

SSD-Based Volumes (Optimized for IOPS)

```
┌──────────────┬──────────┬──────────┬──────────────────────┐
│ Type         │ Max IOPS │ Max MB/s │ Use Case             │
├──────────────┼──────────┼──────────┼──────────────────────┤
│ gp3          │ 16,000   │ 1,000    │ General purpose (DEFAULT)│
│ gp2          │ 16,000   │ 250      │ Legacy general purpose │
│ io2          │ 64,000   │ 1,000    │ Critical DBs, high IOPS │
│ io2 Block Ex │ 256,000  │ 4,000    │ Largest Oracle/SAP   │
│ io1          │ 64,000   │ 1,000    │ Legacy high-perf     │
└──────────────┴──────────┴──────────┴──────────────────────┘
```

HDD-Based Volumes (Optimized for Throughput)

```
┌──────────────┬──────────┬──────────┬──────────────────────┐
│ Type         │ Max IOPS │ Max MB/s │ Use Case             │
├──────────────┼──────────┼──────────┼──────────────────────┤
│ st1          │ 500      │ 500      │ Big data, log processing │
│ sc1          │ 250      │ 250      │ Cold data, lowest cost   │
└──────────────┴──────────┴──────────┴──────────────────────┘

⚠ HDD volumes CANNOT be used as boot/root volumes.
   Only SSD (gp2, gp3, io1, io2) can be root volumes.
```

gp3 vs gp2 — Critical Interview Topic

```
┌──────────────────┬──────────────────┬──────────────────────┐
│ Feature          │ gp2              │ gp3                  │
├──────────────────┼──────────────────┼──────────────────────┤
│ Baseline IOPS    │ 3 IOPS/GB        │ 3,000 (always)       │
│ Max IOPS         │ 16,000           │ 16,000               │
│ IOPS provisioning │ Tied to size     │ Independent          │
│ Max throughput   │ 250 MB/s         │ 1,000 MB/s           │
│ Cost             │ $0.10/GB         │ $0.08/GB (20% cheaper│
└──────────────────┴──────────────────┴──────────────────────┘

gp2 IOPS = 3 × volume size in GB
To get 3,000 IOPS on gp2 → need 1,000 GB = $100/month
To get 3,000 IOPS on gp3 → any size, add $0.005/IOPS = $15/month
gp3 is almost always better. Migrate legacy gp2 to gp3.

gp2 burst:
├── Volumes < 1TB have burst capability (up to 3,000 IOPS)
├── Uses I/O credits (similar to t3 CPU credits)
└── Credit depletes under sustained load → drops to baseline
    (e.g., 100GB gp2 baseline = 300 IOPS. Burst to 3,000 then drops)

io2 / io1:
├── Provision specific IOPS (1 IOPS : 50 GB ratio for io1)
│   io2: 1 IOPS : 1000 GB ratio (much more flexible)
├── Multi-Attach: attach same volume to multiple EC2s (same AZ)
│   Use case: clustered databases (Oracle RAC)
└── 99.999% durability (vs 99.8-99.9% for gp2/gp3)
```

🧩 Snapshots

```
What snapshots are:
Point-in-time backup of an EBS volume stored in S3.
Incremental: first snapshot = full copy.
             Subsequent = only changed blocks.
```

```
Snapshot properties:
├── Stored in S3 (AWS manages the bucket - you don't see it)
├── Incremental (efficient storage)
├── Can be used to create new EBS volumes
├── Can be copied cross-region (for DR)
├── Can be shared with other AWS accounts
└── Billed for total unique blocks across all snapshots

Snapshot flow:
Volume: [Block A][Block B][Block C][Block D]

Snapshot 1 (full):          stores A, B, C, D
Change: block B is modified
Snapshot 2 (incremental):   stores only new B
Change: block D is modified
Snapshot 3 (incremental):   stores only new D

Delete Snapshot 2:
→ Data from Snapshot 2's unique blocks is MOVED to Snapshot 3
→ Snapshot 1 and 3 still fully restorable
→ AWS handles this automatically

⚠ Gotcha: Deleting a snapshot does NOT free all its space
   if subsequent snapshots reference the same blocks.
   AWS moves unique data to next snapshot before deleting.
```

```aws
# Create snapshot
aws ec2 create-snapshot \
  --volume-id vol-0abc123 \
  --description "prod-db before migration" \
  --tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=prod-db-snapshot}]'

# Copy snapshot cross-region (for DR)
aws ec2 copy-snapshot \
  --source-region us-east-1 \
  --source-snapshot-id snap-0abc123 \
  --destination-region us-west-2 \
  --encrypted

# Create volume from snapshot (restore)
aws ec2 create-volume \
  --snapshot-id snap-0abc123 \
  --availability-zone us-east-1a \
  --volume-type gp3

# Automate snapshots with Data Lifecycle Manager
aws dlm create-lifecycle-policy \
  --description "Daily snapshots, keep 7" \
  --state ENABLED \
  --execution-role-arn arn:aws:iam::123:role/dlm-role \
```

```
  --policy-details '{
    "ResourceTypes": ["VOLUME"],
    "TargetTags": [{"Key": "Backup", "Value": "true"}],
    "Schedules": [{
      "Name": "daily",
      "CreateRule": {
        "Interval": 24,
        "IntervalUnit": "HOURS",
        "Times": ["23:45"]
      },
      "RetainRule": {"Count": 7}
    }]
  }'
```

🧩 Encryption

```
What gets encrypted:
├── Data at rest on the volume
├── Data in transit between EC2 and EBS (over the network)
├── All snapshots created from encrypted volume
└── All volumes created from encrypted snapshot

Encryption uses:
├── AWS managed key (aws/ebs) — no cost, default
└── Customer managed KMS key — $1/month + API call costs

Enable encryption by default (account-level):
aws ec2 enable-ebs-encryption-by-default
aws ec2 modify-ebs-default-kms-key-id --kms-key-id alias/my-key

Encrypting an existing unencrypted volume:
(EBS cannot encrypt in-place)

┌──────────────────────────────────────────────────────────┐
│ Unencrypted Volume                                       │
│       ↓ create snapshot                                  │
│ Unencrypted Snapshot                                     │
│       ↓ copy with encryption enabled                     │
│ Encrypted Snapshot                                       │
│       ↓ create volume from encrypted snapshot            │
│ Encrypted Volume ✅                                      │
└──────────────────────────────────────────────────────────┘

Steps:
1. Create snapshot of unencrypted volume
2. Copy snapshot with encryption enabled
3. Create new encrypted volume from encrypted snapshot
4. Stop instance, detach old volume, attach new volume
5. Start instance
```

```
⚠ Encryption is AZ-agnostic for snapshots but volume-specific.
   You can restore an encrypted snapshot in any AZ.

Performance impact of encryption:
Near-zero on Nitro instances (hardware-accelerated AES-256).
Slight impact on older instance types.
```

💬 Short Crisp Interview Answer

"EBS is network-attached block storage for EC2 — like a hard drive that persists independently of the instance. The main volume types are gp3 for general purpose (baseline 3,000 IOPS, independent provisioning, 20% cheaper than gp2), io2 for high-performance databases needing up to 64,000 IOPS with Multi-Attach support, st1 for throughput-heavy sequential workloads like big data, and sc1 for cold data. Snapshots are incremental point-in-time backups stored in S3 — they can be copied cross-region for DR and used to restore volumes. Encryption uses KMS and covers data at rest, in transit, snapshots, and child volumes. You can't encrypt an existing volume in-place — you snapshot it, copy the snapshot with encryption, and restore."

🏭 Real World Production Example

```
Production PostgreSQL database on EBS:

Volume setup:
├── Root volume:  gp3, 100GB   (OS + PostgreSQL binaries)
├── Data volume:  io2, 2TB, 32,000 IOPS provisioned
│                 (PostgreSQL /var/lib/postgresql/data)
└── WAL volume:   io2, 500GB, 8,000 IOPS provisioned
                  (write-ahead log on separate volume = better perf +
recovery)

Snapshot strategy (Data Lifecycle Manager):
├── Every 6 hours: snapshot data + WAL volumes
├── Keep: 48 snapshots (12 days rolling)
├── Copy to us-west-2: daily snapshot (DR)
└── Tag: Backup=true on all volumes

Encryption:
├── All volumes encrypted with customer managed KMS key
├── Key policy allows EC2 instance role to use key
└── Key rotation: annual (automatic)

CloudWatch metrics to monitor:
- VolumeReadOps/WriteOps    → IOPS consumed
- VolumeReadBytes/WriteBytes → throughput
- VolumeQueueLength          → backlog indicator (should be near 0)
- BurstBalance               → for gp2 volumes (alert if < 20%)
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
AZ locked	Volume must be in same AZ as EC2. To move AZ: snapshot → create volume in new AZ
One attachment (mostly)	Can only attach to ONE EC2 at a time, except io1/io2 Multi-Attach
gp2 burst depletion	Small gp2 volumes have low baseline IOPS. Sustained load drains burst credits silently
Snapshot delete ≠ immediate free space	AWS moves unique blocks. Deleting a middle snapshot can be slow
Encryption in-place impossible	Must snapshot → copy with encryption → restore
EBS volume size increase only	Can increase size but CANNOT decrease. Must migrate to smaller volume manually
Fast Snapshot Restore (FSR)	Snapshots have lazy initialization — first I/O to restored block is slow. Enable FSR to pre-warm (costs extra)
Instance EBS bandwidth	Instance has max EBS bandwidth. Many volumes sharing same instance share that bandwidth

- **gp2 burst depletion is a silent failure** — no CloudWatch alarm fires by default when burst credits hit zero. Monitor `BurstBalance` metric and alert below 20%. Small gp2 volumes (under 1TB) are especially vulnerable under sustained load.
- **gp3 throughput advantage is often overlooked** — 1,000 MB/s vs 250 MB/s on gp2 matters significantly for sequential I/O workloads even when IOPS requirements are identical. Always consider throughput alongside IOPS when choosing volume type.

4.3 S3 — Lifecycle Policies, Replication, Presigned URLs

🟢 What It Is in Simple Terms

S3 lifecycle policies automate moving objects between storage classes or deleting them over time. Replication copies objects to another bucket automatically. Presigned URLs give time-limited access to private objects without exposing credentials.

🧩 Lifecycle Policies

```
Lifecycle rule components:
├── Filter: which objects (prefix, tags, or all)
├── Transitions: move to cheaper storage class after X days
└── Expirations: delete objects or versions after X days

Lifecycle rule example:
{
  "Rules": [{
    "ID": "archive-user-uploads",
    "Status": "Enabled",
    "Filter": {"Prefix": "uploads/"},
    "Transitions": [
      {"Days": 30,  "StorageClass": "STANDARD_IA"},
      {"Days": 90,  "StorageClass": "GLACIER"},
      {"Days": 365, "StorageClass": "DEEP_ARCHIVE"}
    ],
    "Expiration": {"Days": 2555}
  }]
}

Valid transition waterfall (must go in this order):
Standard → Standard-IA → One Zone-IA → Intelligent-Tiering
         → Glacier Instant → Glacier Flexible → Deep Archive

⚠ You CANNOT transition from Glacier back to Standard via lifecycle.
   You restore from Glacier — the restored copy is temporary.

Minimum days for transitions:
Standard → Standard-IA: minimum 30 days
Standard-IA → Glacier:  can be same day as IA transition
                        (but IA itself needs 30 days minimum)

Version-aware lifecycle rules:
├── NoncurrentVersionTransitions: move old versions to cheaper storage
├── NoncurrentVersionExpiration: delete old versions after N days
│   (critical for cost control with versioning enabled)
└── ExpiredObjectDeleteMarker: clean up delete markers automatically

Example for versioned bucket:
{
  "NoncurrentVersionTransitions": [
    {"NoncurrentDays": 30, "StorageClass": "STANDARD_IA"},
    {"NoncurrentDays": 90, "StorageClass": "GLACIER"}
  ],
  "NoncurrentVersionExpiration": {"NoncurrentDays": 365},
  "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7}
}
```

```
# Apply lifecycle policy
aws s3api put-bucket-lifecycle-configuration \
```

```
  --bucket my-bucket \
  --lifecycle-configuration file://lifecycle.json
```

🧩 S3 Replication

```
Two types:
├── CRR (Cross-Region Replication): source and dest in different regions
│   Use: compliance, DR, latency reduction for global users
└── SRR (Same-Region Replication): source and dest in same region
    Use: log aggregation, test/prod data sync, compliance copies

Replication requirements:
├── Versioning MUST be enabled on BOTH source and destination
├── IAM role with permissions to read source and write dest
├── One-way by default (A→B does NOT mean B→A)
└── Existing objects NOT replicated automatically
    (use S3 Batch Operations to replicate existing objects)

What IS and ISN'T replicated:
✅ New objects after replication enabled
✅ Object metadata and tags
✅ ACLs
✅ Object lock settings (if replication time control enabled)
❌ Objects that already existed before replication enabled
❌ Objects in Glacier or Deep Archive
❌ Delete markers (by default — can enable DeleteMarkerReplication)
❌ Objects encrypted with SSE-C

Replication Time Control (RTC):
├── SLA: 99.99% of objects replicated within 15 minutes
├── Additional cost
└── Provides replication metrics in CloudWatch

Architecture:
Source Bucket (us-east-1)     Destination Bucket (eu-west-1)
┌──────────────────────────┐   ┌──────────────────────────┐
│ New object PUT           │──►│ Replicated automatically │
│ versioning: enabled      │   │ versioning: enabled      │
│                          │   │                          │
│ IAM Replication Role     │   │ Can have different:      │
│ (cross-account OK)       │   │ - storage class          │
└──────────────────────────┘   │ - encryption key         │
                               │ - ownership              │
                               └──────────────────────────┘
```

```
# Create replication configuration
aws s3api put-bucket-replication \
  --bucket source-bucket \
```

```
  --replication-configuration '{
    "Role": "arn:aws:iam::123:role/s3-replication-role",
    "Rules": [{
      "ID": "replicate-all",
      "Status": "Enabled",
      "Filter": {},
      "Destination": {
        "Bucket": "arn:aws:s3:::dest-bucket",
        "StorageClass": "STANDARD_IA",
        "ReplicationTime": {
          "Status": "Enabled",
          "Time": {"Minutes": 15}
        }
      },
      "DeleteMarkerReplication": {"Status": "Enabled"}
    }]
  }'
```

🧩 Presigned URLs

```
What they solve:
You have a private S3 object. You want to let a user
download it temporarily without:
- Making the bucket public
- Giving them AWS credentials
- Building a proxy server

Solution: Presigned URL
= a time-limited URL with your credentials embedded (HMAC-signed)

How it works:
1. Your server generates presigned URL using AWS SDK
2. URL contains: bucket, key, expiry, your identity, signature
3. User uses URL directly to GET (or PUT) the object
4. S3 verifies signature → allows access
5. After expiry → signature invalid → access denied
```

```python
import boto3

s3 = boto3.client('s3', region_name='us-east-1')

# Presigned URL for download (GET)
url = s3.generate_presigned_url(
    'get_object',
    Params={
        'Bucket': 'my-private-bucket',
        'Key': 'reports/q4-2024.pdf'
    },
```

```
    ExpiresIn=3600  # 1 hour
)
# Returns: https://my-bucket.s3.amazonaws.com/reports/q4-2024.pdf?X-
Amz-...

# Presigned URL for upload (PUT) — user uploads directly to S3
url = s3.generate_presigned_url(
    'put_object',
    Params={
        'Bucket': 'my-private-bucket',
        'Key': f'uploads/{user_id}/{filename}',
        'ContentType': 'image/jpeg'
    },
    ExpiresIn=300  # 5 minutes
)
# Frontend uses this URL with PUT request directly to S3
# Your backend never handles the file bytes!
```

```
⚠ Important nuances:

1. Presigned URL uses credentials of whoever generated it
   - If generated with IAM role, URL is valid as long as
     role session is valid (even if URL expiry is longer!)
   - IAM role sessions typically expire in 1-12 hours
   - If role session expires before URL expiry → URL broken

2. IAM user presigned URLs last as long as specified
   (up to 7 days)

3. Best practice: generate presigned URLs with IAM roles
   but keep expiry shorter than role session duration

Presigned POST (for browser uploads with conditions):
s3.generate_presigned_post(
    Bucket='my-bucket',
    Key='uploads/${filename}',
    Conditions=[
        ['content-length-range', 1, 10485760],  # 1B to 10MB
        {'Content-Type': 'image/jpeg'}
    ],
    ExpiresIn=300
)
More control than presigned PUT:
├── Can restrict: content type, size range, key prefix
├── Browser submits a form POST (not PUT)
└── Returns: URL + form fields (policy, signature, etc.)
```

💬 Short Crisp Interview Answer

"S3 lifecycle policies automate storage class transitions and object expiration. You define rules with prefix/tag filters and specify after how many days to transition objects down the storage tier waterfall — Standard to IA to Glacier to Deep Archive — and when to expire them. For versioned buckets, you also manage noncurrent version transitions and expiration to control costs. S3 Replication (CRR or SRR) copies new objects automatically — versioning must be enabled on both buckets, existing objects are NOT automatically replicated. Presigned URLs let you share time-limited access to private objects — the URL embeds your credentials as a signature. The key gotcha: if the presigned URL is generated by an IAM role, it becomes invalid when the role session expires, even if the URL's own expiry hasn't been reached."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Lifecycle minimum days	Standard → IA requires 30 days minimum
Replication existing objects	Only NEW objects replicated. Use S3 Batch Operations for existing
Delete markers not replicated	Must explicitly enable DeleteMarkerReplication
Glacier objects not replicated	Objects in Glacier/Deep Archive excluded from replication
Presigned URL + IAM role expiry	URL valid only while role session valid
Versioning required for replication	Must enable on BOTH source and destination buckets
SRR ownership	By default, object owner = source account. Enable owner override for cross-account SRR

- **Presigned URL + IAM role = URL expires with the role session** — the URL expiry parameter is an upper bound, not a guarantee. The actual expiry is whichever comes first: the URL expiry or the underlying credential session expiry. Always verify the credential type used to generate the URL when debugging unexpected expiry.
- **CloudFront signed URLs are an alternative for long-lived access** — CloudFront signed URLs use key pairs, not IAM credentials, so they're not affected by IAM session expiry. Use them when URL validity beyond 12 hours is required.
- **S3 Batch Operations is the only way to replicate pre-existing objects** — there is no retroactive replication built into CRR. This is frequently missed when teams enable CRR expecting historical data to appear in the destination.
- **Permanent version deletes are never replicated regardless of settings** — even with `DeleteMarkerReplication` enabled. This protects the destination bucket from destructive operations but means your destination may accumulate versions that no longer exist in the source.

4.4 EFS — Use Cases, Performance Modes, Mount Targets

🟢 What It Is in Simple Terms

EFS (Elastic File System) is AWS's managed NFS (Network File System). Unlike EBS (one volume, one instance), EFS can be mounted by thousands of EC2 instances simultaneously — all seeing the same files. Think of it as a shared network drive in the cloud.

🔍 Why It Exists / What Problem It Solves

EBS is one-to-one (one volume, one EC2). What if you have multiple EC2 instances in an ASG that all need to read/write the same files? Or a container fleet where any container can land on any host? EFS solves this with a POSIX-compliant shared filesystem that multiple instances across multiple AZs can mount simultaneously.

⚙️ How It Works Internally

```
EFS Architecture:

            ┌──────────────────────────────────────────┐
            │              EFS File System             │
            │          (managed, multi-AZ)             │
            └──────────┬──────────┬──────────┬──────────┘
                       │          │          │
          Mount Target  Mount Target  Mount Target
          (AZ-a)        (AZ-b)        (AZ-c)
          10.0.1.x      10.0.2.x      10.0.3.x
                       │          │          │
            ┌──────▼──────┐ ┌──────▼──────┐ ┌──────▼──────┐
            │ EC2/ECS │ │ EC2/ECS │ │ EC2/ECS │
            │  AZ-a   │ │  AZ-b   │ │  AZ-c   │
            └─────────┘ └─────────┘ └─────────┘

All instances see the same filesystem and the same files in real time.

Protocol: NFS v4.1 / NFS v4.0
Access: via mount target (ENI in each AZ's subnet)

Mounting EFS on EC2 (using EFS mount helper — recommended):
sudo mount -t efs -o tls fs-0abc123:/ /mnt/efs
(Handles DNS, TLS encryption in transit automatically)

Manual NFS mount:
sudo mount -t nfs4 \
  -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2 \
```

```
  fs-0abc123.efs.us-east-1.amazonaws.com:/ \
  /mnt/efs
```

🧩 Performance Modes

```
Two performance modes (set at creation, CANNOT change):

1. General Purpose (default) — 99% of use cases
   ├── Lower latency per operation (sub-millisecond)
   ├── Max 35,000 IOPS
   └── Best for: web serving, content management, home directories

2. Max I/O — high parallelism workloads
   ├── Slightly higher latency
   ├── Scales to higher aggregate throughput
   ├── Unlimited IOPS (scales with number of clients)
   └── Best for: big data, media processing with 1000s of instances

⚠ Max I/O is rarely needed today.
   Use Elastic throughput mode instead for most cases.

Three throughput modes:

1. Bursting (legacy default)
   ├── Baseline: 50 KB/s per GB of storage
   │   (1TB = 50 MB/s baseline)
   ├── Burst: up to 100 MB/s (for small filesystems)
   ├── Uses burst credits (like gp2/t3)
   └── ⚠ Small filesystem + heavy I/O = credits deplete fast

2. Elastic (recommended for variable workloads)
   ├── Automatically scales throughput up/down
   ├── Read:  up to 3 GB/s
   ├── Write: up to 1 GB/s
   ├── Pay only for what you use (per GB transferred)
   └── No burst credits to worry about

3. Provisioned
   ├── Specify throughput in MB/s regardless of storage size
   ├── Good for small filesystems with predictable high I/O
   └── More expensive than Elastic for variable workloads
```

🧩 Storage Classes & Lifecycle

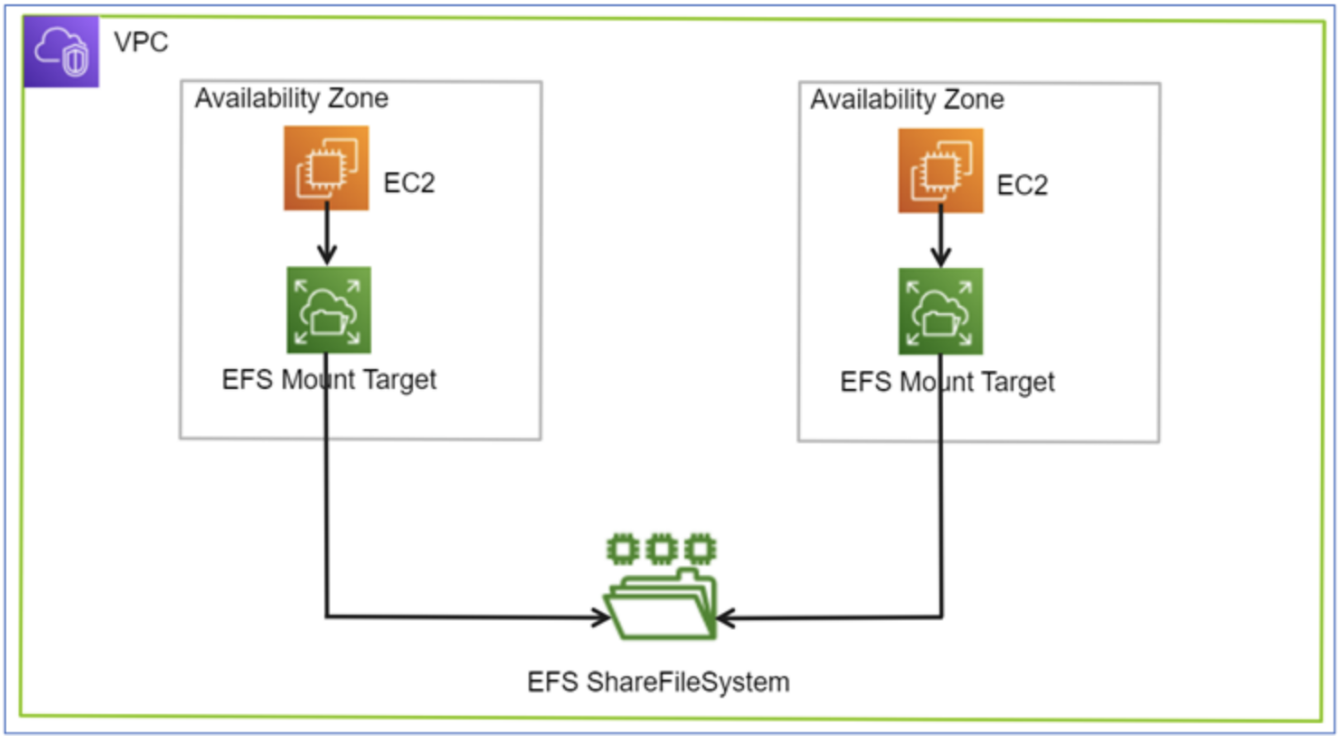
```
EFS Storage Classes:
├── Standard:      frequently accessed files
├── Standard-IA:   infrequently accessed files (lower cost)
```

```
├── One Zone:      single AZ (lower cost, less resilient)
└── One Zone-IA:   single AZ + infrequent access (cheapest)

EFS Intelligent-Tiering lifecycle policy:
{
  "TransitionToIA": "AFTER_30_DAYS",
  "TransitionToPrimaryStorageClass": "AFTER_1_ACCESS"
}
Files not accessed for 30 days → moved to IA
Any access → immediately moved back to Standard

Cost comparison:
Standard:    $0.30/GB/month
Standard-IA: $0.025/GB/month (92% cheaper)
(plus $0.01/GB retrieval fee for IA)
```

🧩 Mount Targets & Security




```
A Mount Target is a network endpoint that allows your EC2 instances in a
specific AZ to access an EFS filesystem over NFS.
├── One per AZ: To allow EC2 instances in an AZ to access EFS, you must
create a mount target in that AZ.
├── Network Interface (ENI): AWS creates an Elastic Network Interface in
your subnet for the mount target
├── The ENI gets an IP from the subnet CIDR. EC2 uses this IP (via DNS) to
communicate with EFS.
└── Security Group: Mount targets have an SG that controls which EC2
instances can connect, usually allowing NFS traffic (TCP 2049) from your
EC2 SG.
```

```
DNS Name: You mount using the filesystem DNS:
`fs-<fs-id>.efs.<region>.amazonaws.com`
AWS resolves this DNS to the mount target in your EC2's AZ.

Network Flow from EC2 to EFS
--> EC2 initiates connection to the EFS DNS (e.g., fs-1234.efs.us-east-
1a.amazonaws.com).
--> DNS resolution: AWS resolves this DNS to the mount target's ENI IP in
the same AZ.
--> Traffic goes through VPC: EC2 sends NFS traffic over TCP port 2049.
--> Security group check: Mount target SG must allow inbound TCP 2049 from
the EC2 SG.
--> NFS protocol: Mount target forwards requests to the EFS filesystem.
--> Filesystem responds: NFS read/write operations return to EC2 over the
same path.

IAM Authorization (EFS Access Points):
├── Access Points define: POSIX user, root directory, permissions
├── Each application gets its own access point (isolation)
└── Can enforce specific UID/GID regardless of what EC2 sends
```

```bash
# Create EFS Access Point
aws efs create-access-point \
  --file-system-id fs-0abc123 \
  --posix-user '{"Uid": 1000, "Gid": 1000}' \
  --root-directory '{
    "Path": "/app1",
    "CreationInfo": {
      "OwnerUid": 1000,
      "OwnerGid": 1000,
      "Permissions": "755"
    }
  }'
```

💬 Short Crisp Interview Answer

"EFS is AWS's managed NFS file system — unlike EBS which is one-to-one, EFS can be mounted simultaneously by thousands of instances across multiple AZs, all seeing the same files in real time. It uses mount targets — one ENI per AZ in your subnet — and communicates over NFS v4.1. For performance, General Purpose mode handles 99% of workloads with the lowest latency. Use Elastic throughput mode for variable I/O rather than the older Bursting mode which depletes credits on small filesystems. EFS Intelligent-Tiering automatically moves infrequently accessed files to Standard-IA at 92% lower cost. The right use cases are shared application state, container persistent storage in ECS/EKS, WordPress content shared across an ASG, or home directories."

🏭 Real World Production Example

```
WordPress ASG with shared media:

Problem: ASG scales to 10 web servers.
         User uploads a profile picture to server 1.
         Next request hits server 3 — image not found!

Solution: EFS for /var/www/html/wp-content/uploads/

┌──────────────────────────────────────────────────────────┐
│              EFS: wp-media-prod                          │
└──────────┬──────────────────┬──────────────────┘
           │                  │                  │
  Mount Target     Mount Target    Mount Target
     AZ-a              AZ-b            AZ-c
       │                  │                  │
  [EC2 Web 1]      [EC2 Web 2]     [EC2 Web 3]
  /var/www/.../    /var/www/.../   /var/www/.../
  uploads/         uploads/        uploads/

All 3 servers see the same /uploads/ directory.
User uploads to any server → visible on all servers.

Mounting in user data:
sudo mount -t efs -o tls,accesspoint=fsap-0abc123 \
  fs-0abc123:/ /var/www/html/wp-content/uploads

Performance: General Purpose mode
Throughput:  Elastic (auto-scales with web traffic)
Storage:     Intelligent-Tiering (old media moves to IA automatically)
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
EFS is more expensive than EBS	~$0.30/GB vs ~$0.08/GB. Only use when shared access is needed
NFS port 2049	Security group on mount target must allow 2049 from EC2 SG
Performance mode is permanent	Set at creation. Cannot change without creating new filesystem
Bursting credits on small filesystems	< 1TB filesystems have very limited burst. Use Elastic mode instead
Windows not supported	EFS is Linux/POSIX only. For Windows shared storage use FSx for Windows
ECS/EKS volumes	EFS works as persistent volumes in ECS and EKS — common interview topic

Gotcha	Detail
TLS encryption in transit	Use EFS mount helper (`-t efs -o tls`) for automatic encryption in transit

---

4.5 S3 Security — Bucket Policies, ACLs, Block Public Access, S3 Object Lock

---

🟢 What It Is in Simple Terms

S3 has multiple overlapping layers of access control. Understanding which layer controls what — and how they interact — is one of the most tested S3 topics in interviews.

⚙️ Access Control Layers (Priority Order)

```
Request to S3 goes through these checks:

1. Block Public Access settings (account or bucket level)
   → If blocking, request denied regardless of policy
   ↓
2. IAM policy (who is making the request?)
   → Principal's identity-based policy
   ↓
3. Bucket policy (resource-based policy)
   → What does the bucket say?
   ↓
4. S3 ACL (legacy access control list)
   → Object or bucket level ACL
   ↓
Access GRANTED if: at least one policy allows AND no explicit deny

⚠ EXPLICIT DENY anywhere = always denied (no exceptions)
   Allow in bucket policy + Deny in SCP = DENIED
```

🧩 Bucket Policies

```
JSON resource-based policy attached to the bucket.
Applies to all objects in the bucket.
Can grant access to other accounts, services, anonymous users.

Structure:
{
  "Version": "2012-10-17",
```

```
  "Statement": [{
    "Sid": "statement-id",
    "Effect": "Allow" or "Deny",
    "Principal": who (IAM user, role, account, *, service),
    "Action": what (s3:GetObject, s3:PutObject, etc.),
    "Resource": which objects (bucket ARN, object ARN),
    "Condition": when (IP, time, encryption required, etc.)
  }]
}
```

Common Bucket Policy Patterns:

```
// 1. Force HTTPS only (deny HTTP)
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": [
    "arn:aws:s3:::my-bucket",
    "arn:aws:s3:::my-bucket/*"
  ],
  "Condition": {
    "Bool": {"aws:SecureTransport": "false"}
  }
}
```

```
// 2. Require KMS server-side encryption on all uploads
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3:::my-bucket/*",
  "Condition": {
    "StringNotEquals": {
      "s3:x-amz-server-side-encryption": "aws:kms"
    }
  }
}

To enforce a specific approved CMK (not just any KMS key), use s3:x-amz-
server-side-encryption-aws-kms-key-id condition key with the exact CMK
ARN. This prevents uploads using the AWS-managed S3 key (aws/s3) or any
other customer key — only your approved CMK is accepted.
```

```
// 3. Restrict to specific VPC endpoint only
{
  "Effect": "Deny",
```

```
  "Principal": "*",
  "Action": "s3:*",
  "Resource": [
    "arn:aws:s3:::my-bucket",
    "arn:aws:s3:::my-bucket/*"
  ],
  "Condition": {
    "StringNotEquals": {
      "aws:sourceVpce": "vpce-0abc123"
    }
  }
}
```

```
// 4. Allow CloudFront only via OAC
{
  "Effect": "Allow",
  "Principal": {"Service": "cloudfront.amazonaws.com"},
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::my-bucket/*",
  "Condition": {
    "StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::123456:distribution/ABCD"
    }
  }
}
```

```
// 5. Cross-account access
{
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::999888777666:root"},
  "Action": ["s3:GetObject", "s3:ListBucket"],
  "Resource": [
    "arn:aws:s3:::my-bucket",
    "arn:aws:s3:::my-bucket/*"
  ]
}
```

🧩 ACLs (Legacy — Mostly Avoid)

```
ACLs are the original S3 access control mechanism.
Predates bucket policies. AWS recommends disabling ACLs
and using bucket policies instead for all new workloads.

Types:
├── Bucket ACL: controls list/read/write on bucket itself
└── Object ACL: controls read/write on individual objects
```

```
Canned ACLs (pre-defined):
├── private (default)
├── public-read
├── public-read-write  ← DANGEROUS: allows anyone to write
├── authenticated-read
├── bucket-owner-read
└── bucket-owner-full-control

⚠ Cross-account object ownership problem:
Account A uploads object to Account B's bucket.
→ By default, Account A OWNS the object (not Account B!)
→ Account B cannot control the object via bucket policy.

Solution — Bucket Owner Enforced (recommended):
aws s3api put-bucket-ownership-controls \
  --bucket my-bucket \
  --ownership-controls 'Rules=[{ObjectOwnership=BucketOwnerEnforced}]'

This disables ACLs entirely.
Bucket owner owns all objects regardless of who uploaded them.
Bucket policy is the only access control mechanism.
```

🧩 Block Public Access ⚠

```
S3 Block Public Access = 4 settings that override everything else:

┌──────────────────────────────────────────────────────────────┐
│ BlockPublicAcls        │ Reject PUT requests that grant      │
│                        │ public access via ACLs              │
├──────────────────────────────────────────────────────────────┤
│ IgnorePublicAcls       │ Ignore existing public ACLs         │
│                        │ (even if ACL says public, block it) │
├──────────────────────────────────────────────────────────────┤
│ BlockPublicPolicy      │ Reject bucket policies that grant   │
│                        │ public access                       │
├──────────────────────────────────────────────────────────────┤
│ RestrictPublicBuckets  │ Even if public policy exists,       │
│                        │ restrict access to AWS services     │
│                        │ and authorized users only           │
└──────────────────────────────────────────────────────────────┘

Two levels:
1. Account level: applies to ALL buckets in account (even future ones)
   This is what AWS enables by default on new accounts.

2. Bucket level: per-bucket settings
   Can be more restrictive than account level, never less.

Best practice: ENABLE ALL 4 at account level.
```

```
Then use bucket policies for controlled cross-account sharing.

⚠ Gotcha: Even with bucket policy allowing public access,
   if RestrictPublicBuckets is enabled, public access is blocked.
   Block settings OVERRIDE policies.

Check account-level BPA:
aws s3control get-public-access-block \
  --account-id 123456789012
```

🧩 S3 Object Lock ⚠ (Critical for Compliance)

```
What it does:
Prevents objects from being deleted or overwritten
for a fixed period or indefinitely.

WORM model: Write Once Read Many
Used for compliance, regulatory requirements, ransomware protection.

Two retention modes:

1. COMPLIANCE mode:
   ├── Object CANNOT be deleted or overwritten by ANYONE
   ├── INCLUDING the root account
   ├── Retention period CANNOT be shortened
   └── Use: Financial records, medical records (HIPAA, SEC 17a-4)

2. GOVERNANCE mode:
   ├── Object CANNOT be deleted by most users
   ├── BUT users with s3:BypassGovernanceRetention CAN override
   └── Use: Internal protection with admin override capability

Retention period:
├── Set per object or via default bucket-level policy
├── Specified as: date (until) or days (from now)
└── COMPLIANCE: cannot shorten. GOVERNANCE: admins can shorten.

Legal Hold:
├── Indefinite lock (no expiry date)
├── Prevents deletion until explicitly removed
├── Independent of retention period
└── Any user with PutObjectLegalHold permission can add/remove

Enabling Object Lock:
├── Must be enabled at BUCKET CREATION (cannot enable later!)
├── Requires versioning (automatically enabled)
└── Lock is applied per version of an object
```

```
# Upload with COMPLIANCE lock
aws s3api put-object \
  --bucket compliance-bucket \
  --key financial-report-2024.pdf \
  --body report.pdf \
  --object-lock-mode COMPLIANCE \
  --object-lock-retain-until-date "2031-12-31T00:00:00Z"
```

```
⚠ Gotcha: If you need Object Lock, you MUST specify it at bucket
   creation. You cannot add it to an existing bucket.
   Plan this in your IaC from day one.
```

💬 Short Crisp Interview Answer

"S3 has layered access control. Block Public Access settings are the highest authority — they override both ACLs and bucket policies. Bucket policies are JSON resource-based policies that are the primary control mechanism — you use them to force HTTPS, require encryption, restrict to VPC endpoints, or grant cross-account access. ACLs are legacy and should be disabled using 'bucket owner enforced' ownership mode for new workloads. S3 Object Lock provides WORM storage — Compliance mode prevents deletion by anyone including the root account and is used for regulatory requirements like SEC 17a-4. Governance mode allows authorized admins to override. Object Lock must be enabled at bucket creation and cannot be added later."

🏭 Real World Production Example

```
Financial services compliance setup:

Bucket: trading-records-prod
├── Object Lock:          ENABLED at creation (COMPLIANCE, 7-year
retention)
├── Versioning:           ENABLED (required by Object Lock)
├── Block Public Access:  ALL 4 ENABLED

Bucket Policy enforces:
1. HTTPS only (deny aws:SecureTransport = false)
2. KMS encryption required on all PutObject
3. Access restricted to specific VPC endpoint only

Result:
- All data encrypted with customer KMS key
- All access over HTTPS only
- Access only from within VPC (no internet)
- Objects immutable for 7 years (compliance mode)
- Even if credentials are leaked:
```

```
    attacker can't delete, can't read from internet,
    and can't overwrite any records
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Block Public Access overrides everything	Even a correctly written public bucket policy is blocked if BPA is enabled
ACL cross-account ownership	Object uploaded by another account is OWNED by that account unless BucketOwnerEnforced
Object Lock at creation only	Cannot enable on existing bucket. Plan in IaC from the start
Compliance mode is absolute	Even root account cannot delete. No exceptions. Be sure before enabling
Deny in bucket policy + Allow in IAM	DENY wins — explicit deny always overrides allow
BPA at account vs bucket level	Account level is the floor. Bucket level can only be more restrictive

- **Object Lock at creation only — cannot be added retroactively** — this is the most operationally painful gotcha. If a compliance requirement emerges after bucket creation, you must create a new bucket and migrate all existing objects. Always enable Object Lock in IaC for any bucket storing regulated data.
- **Compliance mode vs Governance mode is a one-way decision** — you can convert an object from Governance to Compliance mode but not vice versa. You can extend a Compliance retention period but never shorten it. Make this decision carefully before applying to production data.

4.6 S3 — Event Notifications, S3 Select, Multipart Upload

🟢 What It Is in Simple Terms

Three advanced S3 features that unlock event-driven architectures, efficient querying, and reliable large-file uploads.

🧩 S3 Event Notifications

```
What they do:
Automatically trigger downstream actions when S3 objects
```

```
are created, deleted, restored, or replicated.

Supported events:
├── s3:ObjectCreated:*     (PUT, POST, COPY, multipart complete)
├── s3:ObjectRemoved:*     (DELETE, lifecycle expiration)
├── s3:ObjectRestore:*     (Glacier restore initiated/completed)
├── s3:Replication:*       (replication failed/missed SLA)
├── s3:LifecycleTransition
└── s3:IntelligentTiering

Destinations:
├── SNS Topic    → fan-out, email alerts
├── SQS Queue    → queue for processing (most common)
├── Lambda       → direct invocation
└── EventBridge  → most powerful (more routing options, archive, replay)

⚠ Direct notifications (SNS/SQS/Lambda) vs EventBridge:
Direct:       simpler, slightly lower latency, limited routing
EventBridge:  more targets (Step Functions, API GW, etc.),
              event archive and replay, content-based filtering
```

```
# Configure S3 event notifications
aws s3api put-bucket-notification-configuration \
  --bucket my-bucket \
  --notification-configuration '{
    "LambdaFunctionConfigurations": [{
      "LambdaFunctionArn": "arn:aws:lambda:...:my-processor",
      "Events": ["s3:ObjectCreated:*"],
      "Filter": {
        "Key": {
          "FilterRules": [
            {"Name": "prefix", "Value": "uploads/"},
            {"Name": "suffix", "Value": ".jpg"}
          ]
        }
      }
    }],
    "QueueConfigurations": [{
      "QueueArn": "arn:aws:sqs:...:processing-queue",
      "Events": ["s3:ObjectCreated:*"],
      "Filter": {"Key": {"FilterRules": [
        {"Name": "suffix", "Value": ".csv"}
      ]}}
    }]
  }'
```

```
Common event-driven pattern:
User uploads image to S3
    → S3 Event → Lambda
```

```
    → Lambda resizes image to 3 sizes
    → Lambda puts thumbnails back to S3
    → Lambda updates DynamoDB with image metadata

⚠ Gotcha: S3 events are delivered AT LEAST ONCE (not exactly once).
   Your Lambda/processor MUST be idempotent.
   Duplicate events are possible.
```

🧩 S3 Select

```
What it does:
Query (SELECT) data inside S3 objects without downloading the whole
object.
S3 executes the SQL on its side and returns only matching data.

Supported formats:
├── CSV (with/without header)
├── JSON (newline-delimited or document)
├── Parquet (columnar)
└── All can be GZIP or BZIP2 compressed

Supported operations:
SELECT, WHERE, LIMIT — basic SQL subset.
No JOINs, no aggregations in S3 Select.
(Use Athena for complex queries)

Example — 10GB CSV, you need 3 columns of matching rows:
Without S3 Select: download 10GB → filter locally
With S3 Select:    S3 scans internally → returns ~100MB
```

```
aws s3api select-object-content \
  --bucket my-bucket \
  --key data/events-2024.csv.gz \
  --expression "SELECT user_id, event_type, timestamp
                FROM S3Object
                WHERE event_type = 'purchase'
                AND timestamp > '2024-01-01'" \
  --expression-type SQL \
  --input-serialization '{
    "CSV": {"FileHeaderInfo": "USE"},
    "CompressionType": "GZIP"
  }' \
  --output-serialization '{"CSV": {}}' \
  /dev/stdout
```

```
Cost:
Data scanned:  $0.002/GB
Data returned: $0.0007/GB
(vs downloading all 10GB: $0.09/GB data transfer)

⚠ S3 Select vs Athena:
S3 Select: single object, simple filter, lower cost, lower latency
Athena:    multiple objects, complex SQL, JOINs, aggregations,
partitioning
```

🧩 Multipart Upload

```
What it does:
Upload large objects in parts in parallel.
Required for objects > 5 GB.
Recommended for objects > 100 MB.

Benefits:
├── Parallel upload = faster (saturate network bandwidth)
├── Retry individual failed parts (not whole upload)
├── Upload can be paused and resumed
└── Begin upload before you know total object size

How it works:
1. Initiate multipart upload → get Upload ID
2. Upload parts (each part ≥ 5MB, except last part)
3. S3 returns ETag for each part
4. Complete upload (send all ETags) → S3 assembles the object

Part constraints:
├── Part size: 5 MB minimum (except last), 5 GB maximum
├── Max parts: 10,000
└── Max object size: 5 TB (10,000 × 500MB avg)
```

```python
import boto3
from boto3.s3.transfer import TransferConfig

s3 = boto3.client('s3')

# TransferConfig handles multipart automatically
config = TransferConfig(
    multipart_threshold=100 * 1024 * 1024,  # 100MB
    multipart_chunksize=50 * 1024 * 1024,   # 50MB parts
    max_concurrency=10,                      # parallel threads
    use_threads=True
)
```

```
s3.upload_file(
    'large-file.zip',
    'my-bucket',
    'uploads/large-file.zip',
    Config=config
)
```

```
# Manual multipart upload (CLI):

# Step 1: Initiate
UPLOAD_ID=$(aws s3api create-multipart-upload \
  --bucket my-bucket --key large-file.zip \
  --query UploadId --output text)

# Step 2: Upload parts
ETAG1=$(aws s3api upload-part \
  --bucket my-bucket --key large-file.zip \
  --part-number 1 --upload-id $UPLOAD_ID \
  --body part1.bin --query ETag --output text)

ETAG2=$(aws s3api upload-part \
  --bucket my-bucket --key large-file.zip \
  --part-number 2 --upload-id $UPLOAD_ID \
  --body part2.bin --query ETag --output text)

# Step 3: Complete
aws s3api complete-multipart-upload \
  --bucket my-bucket --key large-file.zip \
  --upload-id $UPLOAD_ID \
  --multipart-upload "{
    \"Parts\": [
      {\"PartNumber\": 1, \"ETag\": $ETAG1},
      {\"PartNumber\": 2, \"ETag\": $ETAG2}
    ]
  }"
```

```
⚠ Incomplete multipart uploads — silent cost killer:
If upload is abandoned (client crashes, connection drops),
the parts stay in S3 and you ARE CHARGED for them!
They don't appear as visible objects.

Fix: Add lifecycle rule to abort incomplete uploads:
{
  "AbortIncompleteMultipartUpload": {
    "DaysAfterInitiation": 7
  }
}
```

💬 Short Crisp Interview Answer

"S3 Event Notifications trigger SNS, SQS, Lambda, or EventBridge when objects are created, deleted, or restored. EventBridge is more powerful — it supports more targets, content-based filtering, and event replay. Events are at-least-once delivery, so consumers must be idempotent. S3 Select lets you run SQL-like queries inside S3 objects — CSV, JSON, or Parquet — without downloading the full object, which can reduce data transfer by 99% versus Athena for simple single-object queries. Multipart Upload splits large objects into parallel parts — required over 5 GB, recommended over 100 MB. The critical operational gotcha: abandoned incomplete multipart uploads silently accumulate storage charges. Always add a lifecycle rule to abort incomplete uploads after 7 days."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Event at-least-once	Duplicate events possible. Make consumers idempotent
Abandoned multipart charges	Parts cost money even if upload never completes. Set lifecycle rule
S3 Select vs Athena	S3 Select = single object, simple WHERE. Athena = multi-object, complex SQL
No aggregations in S3 Select	COUNT, SUM, GROUP BY not supported. Use Athena
EventBridge vs direct notifications	EventBridge has richer routing but slightly higher latency (~seconds)
Event notification filter limits	Can filter by prefix OR suffix, but not complex AND conditions per rule

- **EventBridge replay is the killer feature for data pipelines** — the ability to reprocess historical S3 upload events after fixing a bug in the processing Lambda is invaluable. Direct S3 notifications have no equivalent capability. If your pipeline processes financial or compliance-critical data, EventBridge archive + replay is not optional.
- **EventBridge adds ~seconds of latency vs milliseconds for direct notifications** — for real-time processing requirements where sub-second triggering matters, direct S3 → Lambda notifications are still appropriate. EventBridge is not a drop-in replacement when latency is the primary concern.

4.7 EBS — RAID Configurations, io2 Block Express

🟢 What It Is in Simple Terms

RAID lets you combine multiple EBS volumes to get higher performance or redundancy than any single volume can provide. io2 Block Express is AWS's highest-performance EBS tier for mission-critical databases.

🧩 RAID on EBS

```
⚠ RAID in AWS is done at the OS level (software RAID).
   EBS already has built-in AZ-level redundancy.
   RAID on EBS is about PERFORMANCE, not hardware redundancy.

RAID 0 — Striping (performance):
┌──────────┐  ┌──────────┐
│ EBS 1    │  │ EBS 2    │  ← Two separate io2 volumes
│ 32K IOPS│  │ 32K IOPS│
└──────────┘  └──────────┘
     └──────────┬──────────┘
         RAID 0
         64K IOPS combined   ← Double the IOPS
         (data split across both volumes)

Benefits: 2× IOPS, 2× throughput
Risk: If ONE volume fails → ALL data lost (no redundancy)
Use case: Temp processing, scratch space, where speed > safety

RAID 1 — Mirroring (redundancy):
┌──────────┐  ┌──────────┐
│ EBS 1    │  │ EBS 2    │  ← Same data on both
│(primary)│  │ (mirror)│
└──────────┘  └──────────┘

Benefits: Tolerates single volume failure
Cost: 2× storage for same effective capacity
Performance: Write = slightly slower (write to both). Read = same.
Use case: Rarely needed — EBS already replicates within AZ.

⚠ RAID 1 on EBS rarely makes sense:
   EBS is already replicated within the AZ.
   For true redundancy, use Multi-AZ (RDS, Aurora) instead.
   RAID 1 only protects against EBS logical failure, not AZ failure.

RAID 5/6 — NOT recommended by AWS:
RAID 5/6 requires parity writes across volumes.
Each parity write = multiple EBS I/O operations.
This wastes IOPS without meaningful benefit.
AWS explicitly recommends against RAID 5/6 on EBS.
```

🧩 io2 Block Express

```
io2 Block Express = highest performance EBS tier

Specifications:
├── Max IOPS:         256,000 (vs 64,000 for regular io2)
├── Max throughput:   4,000 MB/s (vs 1,000 MB/s)
├── Max volume size:  64 TB
├── Latency:          sub-millisecond, consistent single-digit ms
```

```
  ├── IOPS:GB ratio:    1000:1 (vs 50:1 for io1)
  ├── Durability:       99.999%
  └── Multi-Attach:     up to 16 instances (same AZ)

  Requirements:
  └── Only available on Nitro-based instances
      (R5b, x2idn, x2iedn, x2iezn, C7g, etc.)

  Use cases:
  ├── SAP HANA (requires high sustained IOPS)
  ├── Oracle Exadata workloads in AWS
  ├── SQL Server with extreme I/O needs
  └── Any workload needing > 64,000 IOPS on a single volume

  io2 Block Express vs io2:
  Both are the same "io2" volume type.
  Block Express activates automatically on supported instance types.
  You don't select Block Express explicitly — determined by instance type.
```

💬 Short Crisp Interview Answer

"RAID on EBS is done at the OS level since EBS has no native RAID. RAID 0 stripes across multiple volumes to multiply IOPS and throughput — useful for workloads that need more than a single volume's 64,000 IOPS cap. RAID 1 mirrors for redundancy, but it's rarely justified on EBS since EBS already replicates within the AZ — use Multi-AZ database features instead. io2 Block Express is AWS's top-tier EBS with 256,000 IOPS, 4 GB/s throughput, and sub-millisecond latency — it activates automatically on supported Nitro instance types like R5b when using io2 volumes. It's used for SAP HANA, Oracle RAC, and other enterprise databases with extreme I/O demands."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
RAID 0 = no redundancy	One volume fails = entire RAID array data lost
RAID 5/6 wastes IOPS	Parity operations consume IOPS. AWS explicitly recommends against it
io2 Block Express = automatic	Not a separate volume type. Activated by instance type selection
RAID at OS level	Must configure mdadm or similar. Not an EBS feature
EBS instance bandwidth cap	Even with RAID 0 across many volumes, instance bandwidth is the ceiling

---

4.8 FSx — Windows, Lustre, NetApp ONTAP

🟢 What It Is in Simple Terms

FSx is AWS's family of managed file systems for specialized workloads that need more than EFS can provide. Different FSx flavors speak different protocols for different use cases.

🧩 FSx for Windows File Server

```
What it is:
Fully managed Windows native file system (SMB protocol).
Built on Windows Server. Active Directory integrated.

Key features:
├── Protocol: SMB (Server Message Block) — Windows native
├── Integration: Microsoft Active Directory (LDAP auth)
├── DFS Namespaces: distribute shares across multiple servers
├── Shadow Copies: VSS-based snapshots
├── Storage: SSD or HDD-backed
├── Multi-AZ: active-standby pair (like RDS Multi-AZ)
└── Throughput: up to 2 GB/s, 350,000 IOPS

Use cases:
├── Windows applications needing shared NTFS storage
├── Lift-and-shift Windows workloads to AWS
├── Home directories for Windows users
├── SQL Server on EC2 (shared storage)
└── .NET apps needing UNC paths (\\server\share)

Why NOT use EFS for Windows?
EFS = NFS = POSIX = Linux only
Windows needs SMB = FSx for Windows

Architecture:
┌──────────────────────────────────────────────────────┐
│  FSx for Windows (Multi-AZ)                          │
│  ┌──────────────────┐     ┌──────────────────┐       │
│  │ Active File Svr  │     │ Standby File Svr │       │
│  │ AZ-a             │     │ AZ-b             │       │
│  └──────────────────┘     └──────────────────┘       │
│            │                                          │
│            │ automatic failover (30-45 sec)           │
│  DNS: amznfsx0abc.corp.example.com                   │
└────────────┼─────────────────────────────────────────┘
             │
     Windows EC2 instances mount via SMB:
     net use Z: \\amznfsx0abc.corp.example.com\share
```

🧩 FSx for Lustre

```
What it is:
High-performance parallel file system.
Lustre = Linux cluster file system.
Purpose-built for compute-intensive workloads.

Key features:
├── Protocol: Lustre (POSIX, Linux)
├── Throughput: up to 1 TB/s aggregate
├── IOPS: millions
├── Latency: sub-millisecond
├── S3 integration: lazy load from S3, write back to S3
└── Integration: HPC schedulers (Slurm, LSF, PBS)

Deployment types:
├── Scratch 1/2: temporary, high-burst, no replication
│   Use: short jobs, cost-sensitive, data can be reproduced
└── Persistent 1/2: replicated in AZ, long-running
    Use: ongoing workloads needing durability

S3 integration (linked S3 buckets):
┌──────────────────────────────────────────────────────┐
│ FSx for Lustre ←── lazy load ←── S3 bucket           │
│                                                      │
│ When job accesses file:                              │
│ 1. Check if in Lustre cache                          │
│ 2. If not → fetch from S3 on demand (lazy load)      │
│ 3. Serve from Lustre (fast NVMe speeds)              │
│                                                      │
│ When job writes output:                              │
│ HSM release → write back to S3                       │
└──────────────────────────────────────────────────────┘

This lets you treat S3 as "unlimited slow storage" and
Lustre as "fast scratch space" — data loaded on demand.

Use cases:
├── Machine learning training (thousands of GPU instances)
├── High-performance computing (genomics, CFD, oil & gas)
├── Video rendering and transcoding
├── Financial risk modeling
└── Any workload needing > 1M IOPS or 100+ GB/s throughput

EFS vs FSx for Lustre:
EFS:    low-to-medium performance, simple, Linux and Windows, NFS
Lustre: extreme performance, Linux only, specialized, parallel I/O
```

🧩 FSx for NetApp ONTAP

```
What it is:
Fully managed ONTAP (NetApp's storage OS) on AWS.
The most feature-rich FSx option.

Protocols supported (multi-protocol, simultaneously):
├── NFS (Linux/macOS)
├── SMB (Windows)
└── iSCSI (block storage)

Key ONTAP features on AWS:
├── SnapMirror:       replicate to other AWS regions or on-prem NetApp
├── FlexClone:        instant writable clones of volumes (zero-copy)
├── Deduplication:    automatic data reduction
├── Compression:      inline compression
├── Thin provisioning: allocate more than physical capacity
└── Storage Virtual Machines (SVMs): tenant isolation

Use cases:
├── Lift-and-shift NetApp workloads (familiar ONTAP CLI/API)
├── Applications needing both NFS and SMB simultaneously
├── Oracle databases needing iSCSI block storage
├── DevOps: FlexClone for instant dev/test environment copies
└── Hybrid cloud: SnapMirror to/from on-prem NetApp

FlexClone in DevOps (powerful pattern):
Production DB (2TB) → FlexClone → Dev copy (instant, zero-copy)
├── Dev copy is writable and independent
├── Only changed blocks are stored (copy-on-write)
└── Create 50 dev environments from prod in seconds, not hours
```

🧩 FSx for OpenZFS

```
What it is:
Managed ZFS file system (NFS protocol, Linux/macOS).
High performance with ZFS's data integrity features.

Use cases:
├── Workloads needing ZFS snapshots and clones
├── High-performance NFS for Linux workloads
└── CI/CD environments (instant clones for test isolation)

Performance: up to 1M IOPS, 12.5 GB/s throughput
```

💬 Short Crisp Interview Answer

"FSx is AWS's family of managed specialized file systems. FSx for Windows provides SMB-based NTFS storage with Active Directory integration — the right choice for Windows workloads that need shared

storage, since EFS only speaks NFS. FSx for Lustre is a high-performance parallel file system for HPC and ML training — it can deliver millions of IOPS and over 1 TB/s throughput, and it integrates with S3 for lazy loading, so you treat S3 as unlimited storage and Lustre as a fast cache. FSx for NetApp ONTAP is the most feature-rich option — it supports NFS, SMB, and iSCSI simultaneously on the same filesystem, plus enterprise features like SnapMirror, deduplication, and FlexClone for instant zero-copy volume copies. FlexClone is particularly valuable in DevOps for creating instant dev/test copies of production data."

⚠️ Tricky Edge Cases / Gotchas

Gotcha	Detail
FSx Windows = SMB only	EC2 Linux instances cannot mount FSx for Windows natively
FSx Lustre = Linux only	Lustre client requires Linux. Not for Windows workloads
Lustre Scratch = no durability	Data lost if file server fails. Use Persistent for important data
ONTAP multi-protocol = most expensive	Don't use it if you just need NFS — use EFS instead
FSx vs EFS	EFS = simple scalable NFS. FSx = specialized needs. Don't default to FSx for basic shared storage

---

4.9 Storage Gateway — Types and Use Cases

🟢 What It Is in Simple Terms

Storage Gateway is a hybrid cloud storage service. It's a VM (or hardware appliance) you run in your on-premises data center that appears to your local servers as a standard NAS/SAN/tape library — but actually stores data in AWS S3, EBS, or Glacier under the hood.

🔍 Why It Exists / What Problem It Solves

Enterprises have on-premises applications they can't move to AWS immediately, but they want cloud economics and durability. Storage Gateway bridges on-premises storage protocols (NFS, SMB, iSCSI) to AWS cloud storage — transparently, without changing applications.

⚙️ How It Works Internally

```
On-Premises                          AWS
┌──────────────────────┐             ┌──────────────────────┐
│  Storage Gateway     │             │                      │
│  (VM or appliance)   │◄────HTTPS────►│ S3 / EBS / Glacier  │
```

```
│                        │                │                        │
│ Presents local         │                │ Data stored here       │
│ storage protocol:      │                │ (durable, cheap)       │
│ NFS / SMB / iSCSI      │                └────────────────────────┘
└────────────────────────┘

          ▲
          │ local protocol (fast)
┌─────────▼──────────┐
│ On-prem servers    │
│ (legacy apps,      │
│  backups, VMs)     │
└────────────────────┘

Local cache in the gateway:
├── Hot data: cached locally for low-latency access
└── Cold data: stored in AWS, fetched on demand
```

🧩 Four Types

1. S3 File Gateway

```
Protocol: NFS or SMB
Backend:  S3 bucket
Local cache: most recently accessed files

What it looks like locally:
├── Appears as NFS or SMB file share
├── Files written = objects in S3
└── S3 storage classes can be set per file share

Use cases:
├── On-prem apps that write files → archive to S3
├── Backup files from on-prem to S3
├── Content repositories migrated to S3
└── Transition on-prem NAS data to S3 over time

Example:
On-prem server writes log files to /mnt/logs (NFS)
→ File Gateway transparently stores in S3
→ Lifecycle policy moves to Glacier after 90 days
→ On-prem server never knows data is in cloud
```

---

2. FSx File Gateway

```
Protocol: SMB
Backend:  FSx for Windows File Server
Local cache: most recently accessed files
```

```
Use case:
On-prem Windows users need low-latency access to FSx shares.
Without gateway: all reads go to AWS (high latency for on-prem).
With gateway: local cache serves frequently accessed files fast.
```

---

3. Volume Gateway

```
Protocol: iSCSI (block storage)
Backend:  EBS snapshots stored in S3

Two modes:

Cached Volumes:
├── Primary storage: AWS S3
├── Local cache: frequently accessed data
├── On-prem servers see iSCSI block device
└── Capacity: up to 1 PB across 32 volumes

Stored Volumes:
├── Primary storage: on-premises (local, low latency)
├── AWS: receives scheduled EBS snapshots (backup/DR)
├── Full dataset available locally
└── Capacity: up to 16 TB per volume

Use cases:
├── Cached:  primary storage in cloud, local cache for hot data
├── Stored:  keep data on-prem but backup/DR in AWS
└── Both:    for apps that require iSCSI block protocol
```

---

4. Tape Gateway

```
Protocol: iSCSI VTL (Virtual Tape Library)
Backend:  S3 (active tapes) and Glacier (archived tapes)

What it is:
Replaces physical tape library with virtual tapes.
Appears as a tape library to existing backup software
(Veritas NetBackup, Veeam, Commvault, etc.)

Virtual tapes:
├── Active tape:   in S3 (appears in virtual library)
├── Archived tape: in S3 Glacier or Deep Archive
└── Size:          100 GB to 5 TB per virtual tape

Use case:
Companies with large tape-based backup infrastructure.
```

```
Cannot change backup software or processes.
Want cloud economics instead of physical tape hardware.

Migration path:
Physical tape backup → Tape Gateway → S3/Glacier
No changes to backup software, procedures, or policies.
```

💬 Short Crisp Interview Answer

"Storage Gateway is a hybrid storage service — a VM running on-premises that bridges local storage protocols to AWS. It has four types. S3 File Gateway presents NFS or SMB shares backed by S3, with a local cache for hot data — used to transparently offload on-prem file storage to S3. FSx File Gateway provides local caching for FSx for Windows shares. Volume Gateway presents iSCSI block devices backed by EBS snapshots — in Cached mode, primary data is in AWS with local cache; in Stored mode, primary data is local with AWS as the backup destination. Tape Gateway replaces physical tape libraries with virtual tapes backed by S3 and Glacier — backup software sees a standard tape library and requires no changes."

🏭 Real World Production Example

```
Manufacturing company hybrid storage:

Scenario:
- 50TB of CAD files on on-prem NAS
- Engineers access recent files daily
- Old files rarely accessed but retained 10 years
- Cannot migrate apps to cloud yet

Solution: S3 File Gateway

On-prem: Storage Gateway VM (VMware ESXi)
└── 2TB local SSD cache (recent files served sub-millisecond)

AWS: S3 bucket — cad-files-archive
└── Lifecycle:
    Standard (0-90 days)
    → Standard-IA (90 days)
    → Glacier (1 year)

Engineers access via NFS \\gateway\cad (same as before)
Recent files:  served from local cache (fast, no AWS latency)
Older files:   fetched from S3 on demand (seconds)
Archive files: fetched from Glacier (minutes, scheduled in advance)

Cost savings:
On-prem NAS upgrade (100TB): $150,000 + $20,000/yr maintenance
Storage Gateway + S3:
  50TB Standard-IA: $625/month
```

```
  Gateway instance: $200/month
  Total: ~$10,000/year (vs $20,000+ on-prem)
```

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Gateway is a VM you manage	You maintain the VM (compute, networking). Not fully managed like S3
Local cache sizing	Too small = frequent S3 fetches = high latency. Monitor cache hit rate
Bandwidth dependency	Performance depends on on-prem to AWS bandwidth. Low BW = poor cache-miss experience
Tape retrieval from Glacier	Archived tapes follow Glacier SLAs (hours). Plan archive recalls in advance
File Gateway vs S3 direct	If on-prem app can use AWS SDK, use S3 directly — gateway adds cost and complexity
Volume Gateway snapshots	Automated snapshot schedule must be configured manually. No automatic DR without it

---

🔗 Category 4 — Full Connections Map

```
STORAGE connects to:

S3
├── CloudFront      → CDN origin (OAC for private content)
├── Lambda          → Event notifications, triggered processing
├── SNS/SQS         → Event notification destinations
├── EventBridge     → Richer event routing
├── Athena          → Query S3 data with SQL
├── Glue            → ETL from S3 data
├── KMS             → Server-side encryption keys
├── IAM             → Identity-based + resource-based policies
├── VPC Endpoints   → Private S3 access (Gateway endpoint, free)
├── Replication     → CRR/SRR for DR and compliance
├── Object Lock     → WORM compliance storage
└── Storage Gateway → On-prem to S3 bridge

EBS
├── EC2             → Block storage attached to instances
├── KMS             → Volume encryption
├── CloudWatch      → Volume performance metrics
├── DLM             → Automated snapshot lifecycle
├── ASG             → Root volume template in Launch Template
```

```
  └── RAID (OS level) → Performance scaling beyond single volume

  EFS
  ├── EC2             → NFS mount for shared access
  ├── ECS/EKS         → Persistent volumes for containers
  ├── Lambda          → EFS as Lambda persistent storage
  ├── KMS             → Encryption at rest
  └── IAM + APs       → Fine-grained access control via Access Points

  FSx
  ├── EC2/ECS         → Mount as file storage
  ├── AD (Windows)    → FSx for Windows requires Active Directory
  ├── S3 (Lustre)     → Lazy loading from linked S3 bucket
  ├── SnapMirror      → FSx ONTAP replication to on-prem
  └── Direct Connect  → Low-latency on-prem access to FSx

  Storage Gateway
  ├── S3              → S3 File Gateway, Tape Gateway backend
  ├── EBS Snapshots   → Volume Gateway backend
  ├── Glacier         → Tape Gateway archive backend
  └── VPN/DX          → On-prem connectivity to AWS
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Fact
S3 durability	11 nines (99.999999999%) across 3+ AZs
S3 Standard availability	99.99%
S3 Standard-IA min duration	30 days minimum storage charge
S3 One Zone-IA	Single AZ — AZ failure = data loss
Glacier Deep Archive retrieval	12-48 hours
S3 max object size	5 TB
Multipart required above	5 GB
Multipart recommended above	100 MB
Multipart max parts	10,000
Abandoned multipart	Still charged. Add lifecycle abort rule
gp3 baseline IOPS	3,000 (any size, independent of storage)
gp3 max IOPS	16,000
gp2 IOPS formula	3 × volume size in GB
HDD volumes boot	Cannot be used as root/boot volume
io2 Block Express max IOPS	256,000

Topic	Key Fact
io2 Block Express max throughput	4,000 MB/s
EBS AZ scope	Volume and EC2 must be in same AZ
EBS Multi-Attach	io1/io2 only, same AZ, up to 16 instances
EBS encryption in-place	Impossible — snapshot → copy encrypted → restore
EFS protocol	NFS v4.1
EFS NFS port	2049
EFS performance mode	Set at creation, cannot change
EFS Standard-IA savings	92% cheaper than Standard
EFS vs EBS cost	EFS ~$0.30/GB vs EBS gp3 ~$0.08/GB
Object Lock prerequisite	Must enable at bucket creation — cannot add later
Replication prerequisite	Versioning on BOTH source and destination
Presigned URL + IAM role	URL valid only while IAM role session valid
S3 Select formats	CSV, JSON, Parquet (GZIP/BZIP2 supported)
S3 Select vs Athena	S3 Select = single object, no aggregations. Athena = multi-object, full SQL
Block Public Access	4 settings. Account level overrides bucket level
FSx Windows protocol	SMB
FSx Lustre use case	HPC, ML training, millions of IOPS, TB/s throughput
FSx ONTAP multi-protocol	NFS + SMB + iSCSI simultaneously
FSx Lustre S3 integration	Lazy loads from S3 on demand (treat S3 as unlimited storage)
Storage Gateway types	S3 File, FSx File, Volume (Cached/Stored), Tape
Volume Gateway Cached	Primary in AWS, hot data cached locally
Volume Gateway Stored	Primary on-prem, AWS receives EBS snapshots
RAID 5/6 on EBS	AWS explicitly recommends against — wastes IOPS

Category 4: Storage — Complete Interview Guide
Excluded topics: CodeBuild, CodePipeline, CloudFormation